

LOGO FII

UNIVERSITATEA „ALEXANDRU IOAN CUZA”
DIN IAȘI
FACULTATEA DE INFORMATICĂ

LUCRARE DE LICENȚĂ

ZIDARIO

*Asistent Digital Bazat pe Inteligență Artificială pentru Proiectarea și
Optimizarea Costurilor în Construcții Rezidențiale*

Coordonator Științific:
Decana Lenuta Alboaiei

Absolvent:
Spiridon Roberto

Iași, 2026

Cine sunt?

Mă numesc Spiridon Roberto și sunt student la Facultatea de Informatică din cadrul Universității „Alexandru Ioan Cuza” din Iași. Acești trei ani de facultate au reprezentat o călătorie extraordinară de formare, în care am învățat și am evoluat mai mult decât mi-aș fi putut imagina la începutul acestui drum.

Am pornit de la bazele absolute ale programării, studiind limbaje de nivel scăzut precum Assembly, care m-au ajutat să înțeleg cum funcționează un sistem de calcul. Ulterior, orizontul mi s-a lărgit trecând prin limbaje de nivel înalt și tehnologii moderne, precum Python sau Java. Au fost de folos cursurile de Ingineria Programării, care mi-au structurat gândirea și m-au învățat cum se construiește corect și scalabil un produs software, de la o simplă idee la arhitectură și implementare.

Pe parcursul acestor ani, am avut privilegiul de a întâlni profesori dedicați, care nu doar că ne-au predat materia cu pasiune, dar au dat dovadă de o flexibilitate remarcabilă. S-au adaptat continuu la această nouă eră a Inteligenței Artificiale în care ne aflăm, fiind extrem de deschiși la inovație și încurajându-ne mereu să ieșim din tiparele clasice. Această deschidere m-a inspirat și mi-a dat curajul să abordez pentru lucrarea mea de licență un domeniu complex, încercând să rezolv o problemă reală prin intermediul noilor tehnologii învățate.

Cuprins

Cine sunt?	1
1 Introducere	4
1.1 Contextul Cercetării și Definirea Problemei	4
1.2 Viziunea și Obiectivele ZIDARIO	5
1.3 Modelul de Business și Monetizarea	5
2 Arhitectura Sistemului și Stiva Tehnologică	7
2.1 Stiva Tehnologică (Tech Stack)	7
2.1.1 Frontend (Interfața Utilizatorului)	7
2.1.2 Backend (Logica de Business)	8
2.1.3 Baza de Date și Infrastructura	8
2.2 Arhitectura de Ansamblu	8
2.2.1 Contextul Sistemului (Diagrama C1)	8
2.2.2 Arhitectura de Containere (Diagrama C2)	9
2.3 Modelarea Datelor	9
2.4 Securitatea Sistemului: Decizii Arhitecturale	11
3 Faza 1: Pregătirea Terenului și Configurarea Casei	12
3.1 Prezentare Generală și Obiective	12
3.2 Modulul de Autentificare și Securitate	12
3.2.1 Arhitectura Dual-Token	12
3.2.2 Reînnoirea Sesiunii — Token Rehydration	13
3.2.3 Securitatea Parolelor și Protecția OTP	14
3.2.4 Ordinea Middleware-urilor — Motivație Arhitecturală	14
3.2.5 Protecția Anti-Abuz — Rate Limiting Dual	14
3.3 Normativele Tehnice — Baza Legală a Sistemului	14
3.3.1 Prezentare Generală	14
3.3.2 Strategia de Utilizare: Determinist vs. AI	15
3.4 Modulul Geospațial	16
3.4.1 Arhitectura Serviciului	16
3.4.2 Decizia de a Nu Folosi AI pentru Date Normative	16
3.5 Arhitectura AI: Sistemul Hibrid RAG + CAG	16
3.5.1 De Ce Nu Antrenare sau Fine-Tuning	16
3.5.2 Cache-Augmented Generation (CAG) pentru Date Statice	17
3.5.3 Retrieval-Augmented Generation (RAG) pentru Conținut Dinamic	17
3.5.4 Construcția Prompt-ului Final — Augmentarea Contextului	19
3.6 Streaming SSE	19
3.6.1 Server-Sent Events — Streaming UI Asincron	19

3.6.2	Managementul Memoriei Conversaționale	20
3.7	Wizard-ul de Configurare — Salvarea Progresivă	21
3.7.1	Principiul DB ca Sursă de Adevăr	21
3.7.2	Fluxul Complet al Celor 4 Pași	21
3.8	Izolarea Datelor între Utilizatori — Tenant Security	22
Bibliografie		23

Capitolul 1

Introducere

1.1 Contextul Cercetării și Definirea Problemei

Vreau să încep această lucrare cu o întrebare directă. Dezvoltarea tehnologică creează o multitudine de oportunități în aproape orice sector din industrie, dar de domeniul construcțiilor individuale de ce nu s-a atins cu adevărat nimeni? Dacă ați interacționat vreodată cu acest domeniu, nu ați avut probleme în a înțelege pașii necesari pentru a vă construi propria casă? În a nu depăși bugetul propus? Și, cel mai important, nu ați resimțit lipsa unei transparențe totale asupra costurilor de construcție?

Păi, eu am răspunsul pentru voi — **ZIDARIO**! O platformă care elimină orice barieră tehnică dintre dorința voastră de a vă construi o casă și estimarea precisă a bugetului necesar. În mod tradițional, evaluarea unui plan arhitectural și obținerea unui deviz corect presupun zeci de ore de muncă și implicarea exclusivă a specialiștilor, și chiar și așa, se poate să nu vă satisfaceți toate nevoile până la capăt exact așa cum ați fi vrut.

Proiectul de față îmbină pasiunea mea pentru dezvoltarea web cu algoritmi de inteligență artificială, având ca scop accesul la informații tehnice și financiare în etapa de pre-proiectare a locuințelor cat mai ușor posibil.

Conform datelor publicate de **Institutul Național de Statistică (INS)**, piața construcțiilor rezidențiale din România menține un volum ridicat de activitate, înregistrându-se peste 37.000 de autorizații de construire emise anual pentru clădiri rezidențiale (conform datelor oficiale INS pentru 2023-2024)¹. Cu toate acestea, la nivel european și global, industria se confruntă cu ineficiențe operaționale și financiare severe.

Analizele de specialitate realizate de **McKinsey & Company** indică faptul că proiectele de construcții înregistrează, în medie, depășiri masive ale bugetelor inițiale.²

Dincolo de greșelile umane de planificare, prețurile de producție (*Construction Cost Index*) au explodat, România înregistrând o creștere a costurilor cu aproximativ 137% în perioada 2010–2024³. Principalele cauze care generează aceste deviații sunt estimările manuale imprecise, fragmentarea informațională și lipsa unor instrumente digitale integrate în faza de pre-proiectare. În acest context, platforma propusă adresează direct aceste lacune.

¹Institutul Național de Statistică, https://insse.ro/cms/sites/default/files/com_presa/com_pdf/autoriz12r25.pdf

²McKinsey & Company, *The construction productivity imperative*, <https://www.mckinsey.com/capabilities/operations/our-insights/the-construction-productivity-imperative>

³Eurostat, *Housing in Europe 2024 Interactive Publication*, <https://ec.europa.eu/eurostat/web/interactive-publications/housing>

O estimare potrivită trebuie să țină cont de condițiile de proiectare seismică reglementate prin normativul **P100-1/2013**, de prevederile pentru fundații de suprafață din **NP 112-2014**, de codul pentru structuri din zidărie **CR 6-2013**, precum și de standardele de siguranță la incendiu. Dar, și este un mare „dar”, dorințele clientului împletite cu exigențele minimale ale unei „locuințe convenabile”, cum sunt definite în Legea nr. 114/1996, necesită cunoștințe legislative pe care clienții pur și simplu nu le posedă. Rezultatul este o ruptură informațională gravă între seriozitatea tehnică, nevoile clientului și execuția propriu-zisă, agravată de volatilitatea prețurilor materialelor de pe piață.

1.2 Viziunea și Obiectivele ZIDARIO

Viziunea acestui proiect stă la baza construirii unui asistent digital integrat sub forma unui produs **Software-as-a-Service (SaaS)**. Obiectivul principal este dezvoltarea unui sistem interactiv care preia date de la utilizator (coordonatele geografice ale terenului, poziționarea față de punctele cardinale, tipul de casă dorit) și îi întoarce un plan 2D interactiv, o vizualizare a stadiilor de execuție și, în final, un deviz financiar optimizat.

Obiectivele specifice urmăresc implementarea următoarelor module inteligente:

1. **Modulul de Date Geospațiale:** Integrarea unei hărți web interactive pentru extragerea coordonatelor, esențiale pentru deducerea cerințelor de sol și seismicitate din zona respectivă.
2. **Asistentul Digital Contextual (Arhitectură RAG):** Implementarea unui sistem bazat pe *Retrieval-Augmented Generation*, capabil să interogheze documentele tehnice naționale încorporate. Motivul pentru care nu s-a optat pentru un model lingvistic mare (LLM) generalist simplu este riscul halucinațiilor specifice; sistemul trebuie să ofere o consultanță tehnică strict validată.
3. **Editorul Interactiv 2D:** Oferirea unui spațiu de lucru (canvas) în care utilizatorul poate ajusta geometria locuinței, suprafețele fiind calculate și validate dinamic.
4. **Motorul de Optimizare a Bugetului:** Selecție asistată de AI a materialelor sub constrângere de buget, optimizând raportul calitate-preț pentru a alinia oferta la posibilități financiare ale utilizatorului.

1.3 Modelul de Business și Monetizarea

Arhitectura software propusă depășește stadiul unui simplu exercițiu academic, fiind gândită de la zero ca un **Minimum Viable Product (MVP)** pregătit pentru lansarea pe piață. Soluția rezolvă o problemă a consumatorului final (*B2C*), dar modelul de monetizare este sustenabil prin abordări *B2B*:

- **Generarea de lead-uri calificate:** Utilizatorii își configurează gratuit vizul. Sistemul transformă acest efort într-un „lead” de înaltă calitate, care conține planul prevalidat și bugetul estimat, pachet ce poate fi monetizat către firmele locale de construcții.
- **Integrare Bricolaj (Web Scraping / API):** Devizul este populat cu prețuri extrase dinamic de la marii retaileri din România. Aceasta deschide calea pentru comisioane prin marketing afiliat direct din devizul aplicației.

- **Exportabilitatea Asistentului AI:** Modulul de conversație AI este proiectat ca o componentă de tip *widget* izolată, putând fi licențiat către platformele e-commerce de bricolaj pentru a-i asista pe clienții acestora direct pe site-ul lor.

Capitolul 2

Arhitectura Sistemului și Stiva Tehnologică

Pentru a transpune viziunea platformei ZIDARIO într-un produs funcțional, scalabil și sigur, a fost necesară o selecție riguroasă a tehnologiilor și o proiectare arhitecturală atentă. Acest capitol detaliază deciziile tehnice luate, structura bazei de date și mecanismele de securitate implementate.

Dezvoltarea platformei ZIDARIO a fost structurată în patru faze funcționale independente, fiecare reprezentând un modul complet livrabil. Această abordare incrementală a fost aleasă din două motive: permite validarea fiecărui modul înainte de a construi pe el, și reflectă fluxul natural al unui proiect de construcție rezidențial — de la identificarea terenului până la devizul final.

Faza 1 acoperă configurarea proiectului: identificarea terenului pe hartă, extragerea automată a datelor geospațiale (zonă seismică, adâncime îngheț), asistența AI pentru caracteristicile terenului și configurarea tipului de casă. Faza 2 introduce editorul 2D interactiv în care utilizatorul desenează planul casei, cu calcul automat de suprafețe și validare față de Legea 114/1996. Faza 3 generează devizul complet de materiale pe etape de construcție, cu prețuri actualizate și optimizare de buget asistată de AI. Faza 4 reprezintă direcțiile viitoare: dashboard de proiect management și tracking progres construcție.

Arhitectura tehnică descrisă în acest capitol este comună tuturor fazelor — deciziile de stack, securitate și modelare a datelor au fost luate anticipând cerințele tuturor modulelor, nu doar ale Fazei 1.

2.1 Stiva Tehnologică (Tech Stack)

Aplicația este construită pe o arhitectură modernă de tip Client-Server, utilizând tehnologii capabile să susțină atât interfața grafică complexă (precum planurile 2D și hărțile interactive), cât și volumul mare de date necesar pentru estimările de preț.

2.1.1 Frontend (Interfața Utilizatorului)

Nivelul de prezentare este dezvoltat folosind **React 19**, ajutat prin bundler-ul *Vite* pentru performanță optimă la compilare. De ce React și nu altceva? Datorită ecosistemului matur, a modelului bazat pe componente reutilizabile și a compatibilității native cu bibliotecile necesare proiectului (React Leaflet, React Konva, Framer Motion). Vite înlocuiește bundler-ele clasice precum Webpack, oferind timpi de compilare de până la 10x

mai rapizi prin utilizarea ES Modules native — critic pentru productivitatea în dezvoltare. S-a optat pentru **TypeScript** ca limbaj de bază. De ce Typescript și nu JavaScript vanilla? Elimină o categorie întreagă de erori la runtime prin verificarea statică a tipurilor. Într-un sistem care manipulează date financiare și geometrice, o eroare de tip nedetectată poate produce devize incorecte — TypeScript face aceste erori imposibile la compilare. Design-ul vizual este asigurat de **Tailwind CSS 4**, completat de animații fluide realizate cu *Framer Motion*. Pentru componenta geospațială (selectarea terenului), platforma integrează *React Leaflet* împreună cu *Proj4* pentru manipularea precisă a coordonatelor.

2.1.2 Backend (Logica de Business)

Motorul aplicației este construit pe un runtime **Node.js**, utilizând framework-ul **Express.js** pentru gestionarea rutelor HTTP (API RESTful). Tot codul backend este scris în TypeScript. Node.js permite utilizarea TypeScript pe întregul stack, eliminând context switching-ul între limbaje. Modelul său asincron non-blocant este ideal pentru operațiunile I/O intensive ale Zidario — apeluri simultane către API-uri externe de prețuri, baza de date și serviciul AI. Interacțiunea cu baza de date este abstractizată și securizată prin intermediul **Prisma ORM** (Object-Relational Mapping), care validează schemele de date și previne implicit atacurile de tip SQL Injection.

2.1.3 Baza de Date și Infrastructura

Persistența datelor este asigurată de **PostgreSQL 16**, o bază de date relațională robustă, ideală pentru stocarea legăturilor complexe dintre utilizatori, proiecte și materiale (Bill of Materials - BOM). De ce PostgreSQL din atâtea baze de date? Oferă suport nativ pentru relații complexe între entități (User → Project → BOM → Material), esențiale pentru structura datelor Zidario. Prisma ORM adaugă un strat de siguranță prin query-uri parametrizate care previn implicit SQL Injection, și generează tipuri TypeScript din schema bazei de date. Întregul sistem este containerizat folosind **Docker** și *Docker Compose*, garantând o consistență absolută între mediul de dezvoltare și cel de producție. Serviciile sunt împărțite modular în containere separate pentru baza de date, backend și frontend.

2.2 Arhitectura de Ansamblu

Arhitectura backend-ului urmează șablonul *Controller-Router*, separând clar logica de afaceri de definirea endpoint-urilor API. Această decizie facilitează testarea unitară și extinderea ulterioară a funcționalităților. Arhitectura a fost modelată utilizând standardul de industrie **C4 Model**.

2.2.1 Contextul Sistemului (Diagrama C1)

Diagrama de context (C1) ilustrează granițele platformei și interacțiunile acesteia cu actorii umani și sistemele externe.

Platforma deservește direct utilizatorul final și, opțional, prin licențiere B2B, retailerii de bricolaj. Pentru acuratețea datelor, ZIDARIO comunică cu Google Gemini API (pentru asistentul AI contextual cu arhitectură RAG), ANCPI Geo API (pentru date despre sol) și API-uri ale retailerilor (pentru extragerea prețurilor).

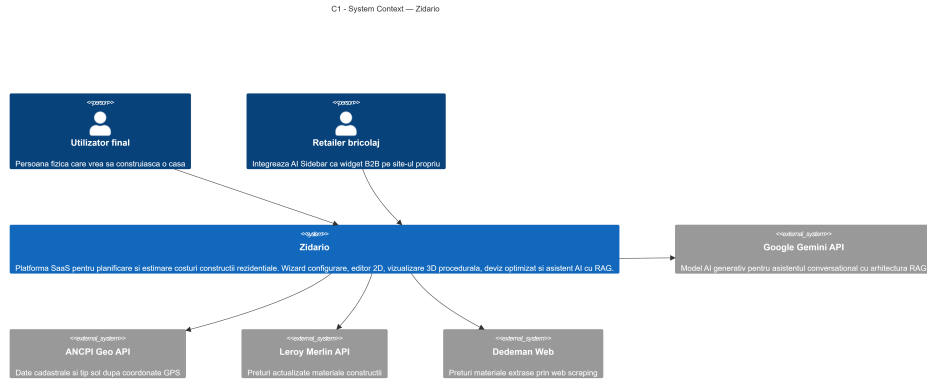


Figura 2.1: Diagrama C1: Contextul Sistemului ZIDARIO

2.2.2 Arhitectura de Containere (Diagrama C2)

La nivel de containere, diagrama C2 detaliază structura internă a aplicației, separată logic pentru a asigura securitatea și mentenabilitatea.

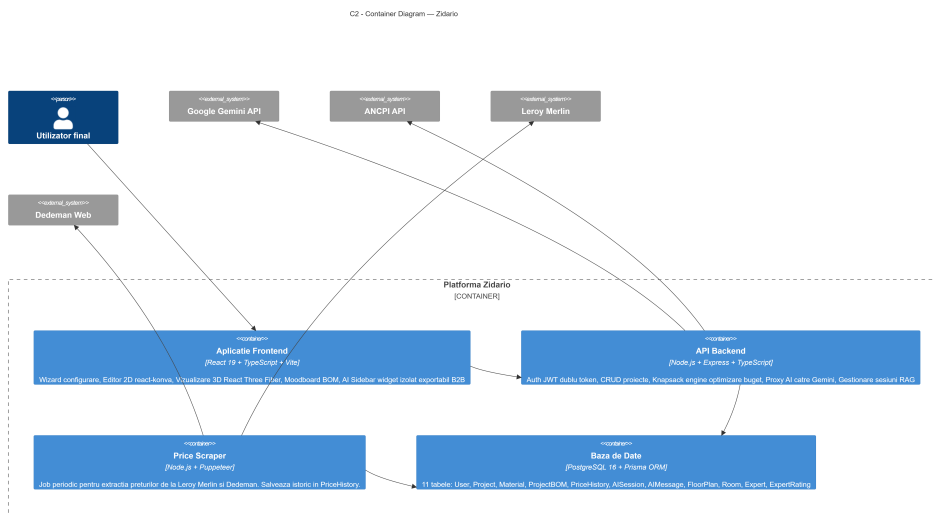


Figura 2.2: Diagrama C2: Container diagram

Fluxul principal de date este gestionat prin rute specifice, cum ar fi `/api/auth` pentru gestiunea identității, `/api/projects` pentru operațiunile CRUD asupra configurațiilor caselor și `/api/materials` pentru interogarea prețurilor. Modulul de asistență AI este gândit ca o componentă izolată (widget), capabilă să comunice independent cu API-ul, sporind astfel potențialul de exportabilitate al platformei (SaaS).

2.3 Modelarea Datelor

Structura bazei de date a fost normalizată pentru a susține generarea dinamică a devizelor (BOM) și a asigura contextul asistentului virtual. Relațiile dintre entități reflectă

direct fluxul de afaceri al aplicației, optimizate prin Prisma ORM.

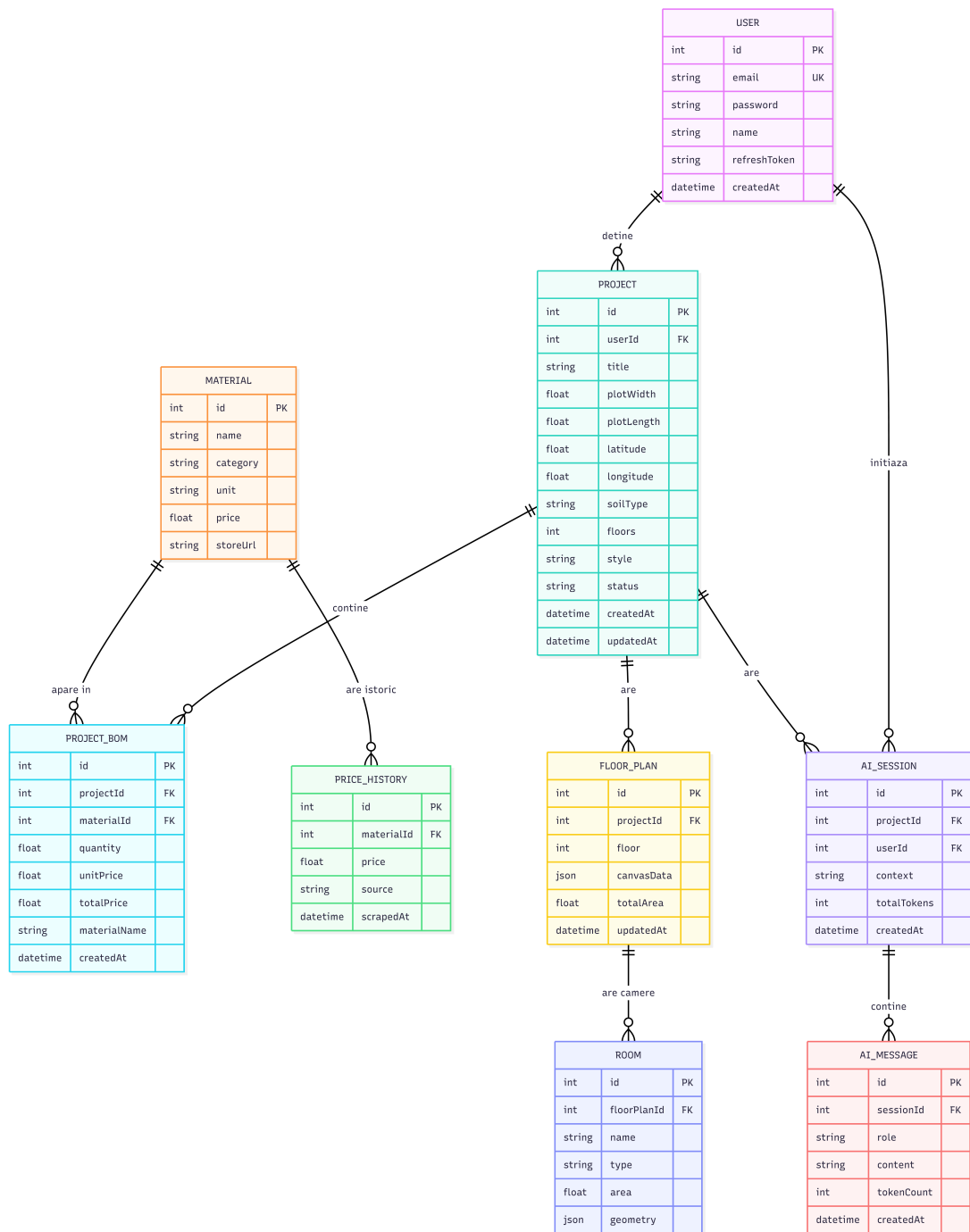


Figura 2.3: Diagrama Entitate-Relatie din baza de date

Structura optimizată cuprinde 9 tabele principale. Entitățile fundamentale sunt:

- **USER:** Stochează credențialele, hash-ul parolei și Refresh Token-ul curent pentru managementul sesiunii.
- **PROJECT:** Reprezintă nucleul configurației realizate de utilizator, stocând variabile critice precum dimensiunile terenului (**plotWidth**, **plotLength**), coordonatele și setările arhitecturale (**soilType**, **floors**). O instanță de utilizator poate avea o relație *One-to-Many* cu entitatea proiect.

- **MATERIAL** și **PRICE_HISTORY**: Tabelul de materiale acționează ca un catalog predefinit sau populat prin scraping. Tabelul de istoric stochează evoluția prețurilor în timp.
- **PROJECT_BOM (Bill of Materials)**: O tabelă de joncțiune ce leagă materialele necesare de un anumit proiect, calculând cantitatea exactă și prețul total per material.
- **AI_SESSION** și **AI_MESSAGE**: Persistă starea și istoricul conversațiilor asistentului digital RAG. Prin legarea directă a sesiunii de un proiect, sistemul evită depășirea limitelor de context (token-uri), injectând un prompt dinamic bazat exclusiv pe datele curente ale proiectului.
- **FLOOR_PLAN** și **ROOM**: Gestionează datele geometrice ale casei. Tabelul **FLOOR_PLAN** salvează structurile desenate în editorul 2D sub forma unui payload JSON (**canvasData**), esențial pentru calculele automate de suprafață.

2.4 Securitatea Sistemului: Decizii Arhitecturale

Deoarece platforma gestionează informații corelate cu bugete și preferințe personale, a fost adoptat principiul *Security by Design*. S-a stabilit utilizarea unui mecanism **Stateless** pentru sesiuni, fundamentat pe tehnologia **JSON Web Tokens (JWT)**, dublat de criptarea ireversibilă a parolelor cu algoritmul **bcrypt**.

Principala decizie arhitecturală pe ramura de securitate a fost separarea responsabilităților token-urilor în *Access Token* (durată scurtă, păstrat strict în memoria volatilă) și *Refresh Token* (durată lungă, izolat prin mecanisme de browser tip **HTTP-Only cookie**). Această segregare asigură imunitatea sistemului la atacurile de tip XSS (Cross-Site Scripting) fără a degrada experiența utilizatorului. Detaliile tehnice de implementare, inclusiv scrierea *middleware-urilor* pe backend și orchestrarea *interceptorilor Axios* pe partea de frontend, vor fi aprofundate în cadrul **Capitolul 3** al acestei lucrări.

Capitolul 3

Faza 1: Pregătirea Terenului și Configurarea Casei

3.1 Prezentare Generală și Obiective

Prima fază a platformei ZIDARIO acoperă fluxul complet de pre-proiectare: de la identificarea și analiza terenului, până la stabilirea configurației structurale a viitoarei locuințe. Această fază reprezintă fundamentul pe care se construiesc toate modulele ulterioare — editorul 2D (Faza 2) și motorul de deviz (Faza 3) depind în totalitate de datele colectate și validate în Faza 1.

Obiectivele tehnice ale acestei faze sunt:

1. Implementarea unui sistem de autentificare securizat bazat pe arhitectura Dual-Token JWT
2. Construirea unui flux wizard în 4 pași cu persistență progresivă în baza de date
3. Extragerea automată a datelor geospațiale din coordonate GPS
4. Implementarea asistentului AI Zidario cu arhitectură hibridă RAG + CAG și streaming SSE
5. Validarea configurației structurale față de normativele tehnice românești în vigoare

3.2 Modulul de Autentificare și Securitate

3.2.1 Arhitectura Dual-Token

Sistemul de autentificare implementează o strategie bazată pe două token-uri JWT cu responsabilități și cicluri de viață distincte.

Access Token-ul are un TTL (Time To Live) de 15 minute și este returnat clientului exclusiv prin corpul răspunsului JSON. Clientul îl stochează în memoria volatilă a aplicației React — o variabilă de stare, nu localStorage sau sessionStorage. Această decizie este deliberată: memoria volatilă nu este accesibilă JavaScript-ului injectat malițios printr-un atac XSS. Cel mai mare dezavantaj — token-ul se pierde la refresh de pagină — este rezolvat transparent prin mecanismul de reînnoire automată descris în secțiunea următoare.

Refresh Token-ul are un TTL de 7 zile și este trimis clientului exclusiv printr-un cookie cu attributele `HttpOnly`, `Secure` și `SameSite=Strict`. Atributul `HttpOnly` interzice orice acces JavaScript la cookie — codul `document.cookie` nu va returna niciodată Refresh Token-ul, indiferent de originea scriptului. Atributul `SameSite=Strict` elimină vectorul de atac CSRF (Cross-Site Request Forgery), refuzând trimiterea automată a cookie-ului în request-uri originate de pe domenii externe.

```
1 // authController.ts - generarea si trimiterea token-urilor la
   ↪ login
2 const accessToken = jwt.sign(
3   { id: user.id, email: user.email },
4   process.env.JWT_SECRET!,
5   { expiresIn: '15m' }
6 );
7
8 const refreshToken = jwt.sign(
9   { id: user.id },
10  process.env.JWT_REFRESH_SECRET!,
11  { expiresIn: '7d' }
12 );
13
14 // Refresh Token - cookie izolat de JavaScript
15 res.cookie('refreshToken', refreshToken, {
16   httpOnly: true,           // JavaScript nu poate citi acest cookie
17   secure: true,             // transmis doar pe HTTPS
18   sameSite: 'strict',      // blocat in request-uri cross-site
19   maxAge: 7 * 24 * 60 * 60 * 1000
20 });
21
22 // Access Token - body JSON - stocat in memorie de client
23 res.json({ accessToken, user: { id: user.id, email: user.email }
   ↪});
```

3.2.2 Reînnoirea Sesiunii — Token Rehydration

La refresh-ul paginii, Access Token-ul din memorie se pierde. Mecanismul de reînnoire automată, implementat prin interceptorii Axios pe frontend, rezolvă această problemă transparent pentru utilizator.

Fluxul este următorul: interceptorul Axios detectează un răspuns cu status 401, apelează automat endpoint-ul `/api/auth/refresh`, serverul citește Refresh Token-ul din cookie, îl validează față de hash-ul stocat în baza de date, generează un nou Access Token și îl returnează. Interceptorul reface request-ul original cu noul token — utilizatorul nu observă întreruperea.

Validarea Refresh Token-ului față de baza de date, și nu doar față de semnătura JWT, oferă un avantaj critic de securitate: token-ul poate fi revocat oricând prin ștergerea înregistrării din baza de date, util în scenarii de compromitere a contului sau logout forțat de administrator.

3.2.3 Securitatea Parolelor și Protecția OTP

Parolele sunt stocate exclusiv ca hash-uri bcrypt cu salt factor 10. Bcrypt este o funcție de derivare a cheilor (Key Derivation Function) deliberat lentă, rezistentă la atacuri de tip brute-force și Rainbow Tables. Salt-ul unic per parolă garantează că două parole identice produc hash-uri diferite în baza de date.

Mecanismul de recuperare prin OTP (One-Time Password) implementează două măsuri suplimentare. Codul de 6 cifre este stocat hash-uit cu SHA-256 — dacă baza de date este compromisă, atacatorul nu obține codurile active. Răspunsul endpoint-ului este identic indiferent dacă adresa de email există sau nu în sistem, prevenind atacurile de tip User Enumeration, prin care un atacator ar putea construi lista utilizatorilor platformei testând adrese de email.

3.2.4 Ordinea Middleware-urilor — Motivație Arhitecturală

Ordinea înregistrării middleware-urilor în `index.ts` nu este arbitrară. Fiecare poziție are o justificare de securitate sau performanță:

`helmet()` este primul, deoarece setează headerele de securitate HTTP pe orice răspuns, indiferent de ce urmează. `cors()` urmează imediat, respingând rapid request-urile cu Origin neautorizat înainte ca serverul să consume resurse parsând body-ul. `express.json()` vine după CORS, deoarece parsarea body-ului JSON este o operație cu cost computațional — nu are sens să o efectuezi pentru un request care va fi oricum respins. `rateLimiter` este plasat înaintea `verifyToken` din același motiv: verificarea semnăturii JWT implică operații criptografice costisitoare. Un atacator care face flood cu token-uri invalide ar consuma CPU pentru criptografie înainte să fie blocat — rate limiter-ul acționează ca un gardian ieftin pus cât mai devreme în lanț.

3.2.5 Protecția Anti-Abuz — Rate Limiting Dual

Sistemul implementează o strategie de rate limiting pe două niveluri pentru a echilibra protecția cu accesibilitatea utilizatorilor legitimi.

Limiter-ul global restricționează numărul total de request-uri per IP per fereastră de timp, protejând serverul împotriva atacurilor DDoS generice.

Limiter-ul per email + IP se aplică exclusiv endpoint-urilor de autentificare. Strategia duală previne un scenariu specific: un atacator care vizează un cont anume poate folosi rețele proxy cu IP-uri în rotație. Limitarea per email blochează atacul chiar dacă IP-ul se schimbă. Simultan, limitarea per IP nu penalizează utilizatorii legitimi care împart același IP, cum ar fi angajații dintr-o rețea corporativă.

3.3 Normativele Tehnice — Baza Legală a Sistemului

3.3.1 Prezentare Generală

Un element esențial care diferențiază ZIDARIO de un simplu configurator de locuințe este ancorarea în cadrul normativ și legislativ românesc în vigoare. Toate regulile pe care sistemul le aplică automat — numărul maxim de etaje, adâncimea fundației, suprafețele minime ale camerelor — provin din documente normative oficiale, nu din estimări empirice sau din cunoștințele generale ale modelului AI.

Normativele utilizate în Faza 1 sunt grupate în trei categorii funcționale:

Grupa 1 — Normative structurale și geotehnice

- **P100-1/2013 — Cod de proiectare seismică:** definește zonarea seismică a teritoriului României. Din acesta a fost extrasă Anexa A, care conține valorile accelerației de vârf ale terenului (a_g) și perioada de colț (T_c) pentru fiecare județ. Aceste valori sunt stocate în fișierul `seismic-zones.json` și folosite pentru determinarea automată a zonei seismice din coordonatele GPS ale terenului, precum și pentru calculul numărului maxim de etaje permis.
- **NP112-2014 — Normativ pentru proiectarea fundațiilor de suprafață:** definește adâncimea minimă de fundare în funcție de adâncimea de îngheț a solului, diferențiată pe regiuni geografice. Din acesta a fost extrasă Anexa B cu valorile per județ, stocate în `frost-depth.json`. Această valoare este prezentată utilizatorului în Pasul 3 al wizard-ului ca restricție minimă obligatorie pentru fundație.
- **CR6-2013 — Cod de proiectare pentru structuri din zidărie:** furnizează regulile structurale pentru construcțiile din zidărie, inclusiv limitările de înălțime în funcție de zona seismică și tipul de sol. Aceste reguli au fost formalizate în matricea din `floor-rules.json`.

Grupa 2 — Legislație privind locuințele

- **Legea 114/1996 — Legea locuinței:** definește în Articolul 5 suprafețele utile minime obligatorii pentru fiecare tip de încăpere dintr-o locuință convenabilă. Aceste valori sunt folosite în Faza 2 pentru validarea planului 2D desenat de utilizator, dar sunt preîncărcate în sistemul CAG încă din Faza 1 pentru a fi disponibile asistentului AI.

Grupa 3 — Normative de referință pentru asistentul RAG

- **NP112-2014 și P100-1/2013** sunt procesate integral prin pipeline-ul de indexare RAG — nu doar valorile tabelate, ci și textul explicativ al articolelor. Aceasta permite asistentului să citeze și să explice motivele tehnice din spatele restricțiilor, nu doar să le enunțe.

3.3.2 Strategia de Utilizare: Determinist vs. AI

Normativele sunt utilizate în două moduri distincte, cu o separare clară de responsabilități:

- **Utilizarea deterministă:** valorile numerice critice (zona seismică, adâncimea de îngheț, numărul maxim de etaje) sunt extrase o singură dată manual din normative, stocate în fișiere JSON și folosite direct în logica aplicației. Aceste valori nu trec niciodată printr-un model AI — sunt constante tehnice cu valoare legală.
- **Utilizarea prin RAG:** textul explicativ al normativelor este vectorizat și indexat în `pgvector`. Asistentul AI îl accesează la runtime prin căutare semantică pentru a explica de ce există o restricție, citând articolul exact. Modelul nu calculează — explică.

Această separare elimină riscul de halucinație pe date critice de siguranță, menținând în același timp valoarea adăugată a explicațiilor contextuale oferite de AI.

3.4 Modulul Geospațial

3.4.1 Arhitectura Serviciului

Modulul geospațial rezolvă problema extragerii automate a datelor tehnice normative dintr-o pereche de coordonate GPS. Utilizatorul dă click pe hartă — sistemul returnează automat județul, zona seismică conform P100-1/2013 și adâncimea minimă de îngheț conform NP112-2014, fără nicio interacțiune suplimentară din partea utilizatorului.

Serviciul este compus din trei straturi independente. Primul este reverse geocoding-ul prin API-ul public Nominatim (OpenStreetMap)¹, care transformă coordonatele GPS în denumirea județului. Nominatim a fost ales față de Google Maps Geocoding API din motive de cost zero și absența cheii API, suficient pentru granularitatea județeană necesară aplicației.

Al doilea strat este matching-ul în JSON-uri normative locale: județul returnat de Nominatim este folosit ca cheie de căutare în fișierele `seismic-zones.json` și `frost-depth.json`, care conțin valorile extrase din normativele P100-1/2013 și NP112-2014.

Al treilea strat este calculul regulilor deterministe pentru numărul maxim de etaje: fișierul `floor-rules.json` conține matricea combinată zonă seismică $\times$ tip sol $\rightarrow$ număr maxim de etaje permis.

3.4.2 Decizia de a Nu Folosi AI pentru Date Normative

Aceasta este una dintre deciziile arhitecturale fundamentale ale platformei și merită justificată explicit.

Datele precum zona seismică a unui județ sau adâncimea minimă de îngheț sunt valori deterministe cu valoare legală. Acestea nu se interpretează — sunt constante tehnice extrase din documente normative oficiale. Delegarea acestor calcule unui model de limbaj ar introduce un risc inacceptabil: modelele LLM pot halucina valori numerice, în special când sunt întrebat despre date specifice dintr-un domeniu tehnic restrâns. Consecința unui deviz de fundație calculat pe o zonă seismică greșită poate fi o structură subdimensionată.

Prin JSON deterministic, valoarea este întotdeauna exactă și auditabilă. Dacă normativul P100 este revizuit, modificarea înseamnă editarea unui singur rând în fișier, fără reantrenare sau migrare de bază de date. AI-ul intervine abia în pasul următor: explică de why există aceste limite, citând articolele din normativ — nu le calculează.

3.5 Arhitectura AI: Sistemul Hibrid RAG + CAG

3.5.1 De Ce Nu Antrenare sau Fine-Tuning

Înainte de a descrie arhitectura implementată, este necesar să clarificăm de ce alternativele standard nu sunt potrivite pentru cazul de utilizare al platformei ZIDARIO.

Antrenarea unui LLM înseamnă modificarea a miliarde de parametri interni ai rețelei neuronale prin gradient descent, pe sute de miliarde de tokeni, pe clustere de mii de GPU-uri, timp de săptămâni. Costul se măsoară în zeci de milioane de dolari² Această opțiune

¹Serviciul oficial Nominatim: <https://nominatim.openstreetmap.org>

²Sevilla et al. (2022), "Compute Trends Across Three Eras of Machine Learning": <https://arxiv.org/abs/2202.05924>.

nu este relevantă pentru un produs MVP.

Fine-tuning-ul antrenează în continuare un model existent pe un set de date specific domeniului, cu costuri mai accesibile. Cu toate acestea, are o limitare fundamentală pentru cazul nostru: cunoștințele sunt baked in în parametrii modelului. Dacă zona seismică a unui județ se modifică după revizuirea normativului P100, modelul fine-tunat va returna în continuare valoarea veche până la un nou ciclu de fine-tuning costisitor. Un aspect critic adesea ignorat este că fine-tuning-ul îmbunătățește stilul și formatul răspunsurilor, dar nu îmbunătățește acuratețea faptelor numerice — un model fine-tunat poate în continuare halucina valori, deci JSON-urile deterministe și RAG-ul sunt necesare oricum.

Tabelul următor prezintă cei patru factori care determină calitatea unui sistem AI pe un domeniu specific, în ordinea impactului real:

Factor	Abordare ZIDARIO
Calitatea modelului de bază	Google Gemini 2.5 Pro
Calitatea contextului injectat	RAG + CAG [Lewis et al., 2020; Gao et al., 2023]
Calitatea prompt engineering-ului	Prompt structurat cu reguli absolute
Fine-tuning pe domeniu	Nu este aplicat în MVP

Tabela 3.1: Factori determinanți ai calității sistemului AI pe domeniu specific (surse: [Lewis et al., 2020](#), [Gao et al., 2023](#), [Sevilla et al., 2022](#)).

3.5.2 Cache-Augmented Generation (CAG) pentru Date Statice

CAG este tehnica prin care date cu caracter static sunt preîncărcate o singură dată în memoria serverului la pornire și injectate direct în fiecare prompt de sistem, fără a fi căutate la runtime prin similaritate vectorială.

Modulul `normativeCache.ts` implementează pattern-ul Singleton pentru această funcționalitate: la prima invocare, datele sunt citite de pe disc și stocate într-o variabilă de modul. La orice invocare ulterioară, variabila deja populată este returnată instant. Justificarea este că citirea fișierelor de pe disc implică operații I/O cu latență de ordinul milisecundelor, în timp ce citirea din RAM are latență de ordinul nanosecundelor. Pentru date care nu se modifică pe durata rulării serverului, caching-ul în memorie este optimal.

Datele selectate pentru CAG sunt exclusiv constante matematice și legale: valorile din P100-1/2013 (zone seismice per județ), NP112-2014 (adâncime îngheț per județ), regulile de etaje per zonă seismică și tip sol, și suprafețele minime din Legea 114/1996. Acestea nu au nuanțe contextuale, iar o eroare de retrieval pentru ele ar fi inacceptabilă — injectarea directă în prompt garantează zero erori de citire a indicilor seismici.

3.5.3 Retrieval-Augmented Generation (RAG) pentru Conținut Dinamic

RAG este un cadru arhitectural format din două faze distincte care permit unui model de limbaj să răspundă pe baza unor documente specifice, fără a fi antrenat pe acestea.

Faza A — Indexarea (offline, rulată o singură dată prin `seedNormatives.ts`)

Această fază procesează documentele normative PDF și le pregătește pentru căutare semantică, printr-un pipeline în patru etape:

1. Extragerea textului brut din fișierele PDF ale normativelor folosind biblioteca `pdf-parse`.
2. Segmentarea (chunking) textului în fragmente de aproximativ 800 de cuvinte — dimensiune aleasă ca un compromis deliberat: chunk-uri prea mici pierd contextul (un articol de lege fără capitolul în care apare poate fi ambiguu), chunk-uri prea mari depășesc fereastra de context și diluează relevanța la căutare.
3. Vectorizarea (embedding): fiecare fragment este transformat într-un vector de 768 de numere reale prin modelul `text-embedding-004` de la Google, unde fragmentele cu înțeles semantic similar produc vectori apropiați geometric.
4. Stocarea vectorilor în tabelul `NormativeChunk` din PostgreSQL prin extensia `pgvector`.

```

1 // seedNormatives.ts - pipeline de ingestie si vectorizare
2 async function seedNormative(filePath: string, source: string,
3   ↪ agent: string) {
4   const { text } = await pdfParse(fs.readFileSync(filePath));
5   const chunks = splitIntoChunks(text, 800);
6
7   for (const chunk of chunks) {
8     const embedding = await embeddingService.embed(chunk.content)
9     ↪;
10    await prisma.normativeChunk.create({
11      data: {
12        source,
13        agent,           // "geotehnic" / "seismic" / "legal"
14        chapter: chunk.title,
15        content: chunk.content,
16        embedding,
17      }
18    });
19    await sleep(100); // respectam rate limit-ul Gemini API (15
20    ↪ req/min, tier gratuit)
21  }
22 }

```

Faza B — Interogarea (online, la fiecare mesaj al utilizatorului)

Când utilizatorul trimite o întrebare asistentului Zidario, are loc următorul flux: întrebarea este transformată în vector prin același model `text-embedding-004`, se calculează distanța cosinus între vectorul întrebării și toți vectorii din `NormativeChunk`, sunt returnate top-5 fragmente cu similitudinea cea mai mare, iar acestea sunt injectate în prompt-ul final trimis modelului Gemini.

```

1 // ragService.ts - cautare semantica in pgvector
2 async function searchRelevantChunks(
3   question: string,
4   agent: string
5 ): Promise<NormativeChunk[]> {
6   const questionEmbedding = await embeddingService.embed(question
7     ↪);
8   return prisma.$queryRaw `

```

```

9      SELECT *, 1 - (embedding <=> ${questionEmbedding}::vector) AS
      ↪ similarity
10     FROM "NormativeChunk"
11     WHERE agent = ${agent}
12     ORDER BY similarity DESC
13     LIMIT 5
14     ' ;
15 }

```

Operatorul `<=>` din pgvector calculează distanța cosinus între doi vectori. Expresia `1 - distanță` transformă distanța în scor de similitudine — valoarea 1 înseamnă vectori identici, 0 înseamnă vectori complet diferiți. Indexul `ivfflat` (Inverted File with Flat compression) face această căutare sub-liniară în complexitate, esențial pe seturi mari de chunk-uri.

Optimizarea prin izolarea pe agenți specializați

O decizie arhitecturală importantă este câmpul `agent` din tabelul `NormativeChunk`. Fiecare chunk este etichetat cu agentul specializat care îl utilizează: geotehnic pentru NP112 și datele de sol, seismic pentru P100 și structură, legal pentru Legea 114 și suprafețe minime.

Când utilizatorul pune o întrebare despre adâncimea fundației, sistemul caută exclusiv în chunk-urile agentului geotehnic. Aceasta elimină zgomotul semantic: paragrafe din normativele de instalații electrice nu vor apărea în context când se discută despre fundații, reducând semnificativ rata de halucinație a modelului.

3.5.4 Construcția Prompt-ului Final — Augmentarea Contextului

Prompt-ul final care ajunge la Gemini este asamblat de `agentOrchestrator.ts` din patru componente distincte: instrucțiunile de comportament cu reguli absolute (citează normativul și articolul exact, nu inventa valori numerice, recomandă inginer autorizat când nu există certitudine), datele proiectului curent (județ, zonă seismică, tip sol), tabelele CAG cu valorile normative statice și chunk-urile RAG cu articolele relevante din normative.

Diferența față de un prompt simplu este ilustrată prin contrast. Un LLM generic întrebat „*ce adâncime de fundație îmi trebuie?*” răspunde vag: „*depinde de mai mulți factori*”. Zidario cu contextul complet răspunde: „*pentru terenul din județul Suceava, pe sol argilos, adâncimea minimă de fundare este 110cm conform NP112-2014, Art.4.3. Pe sol argilos se recomandă un spor de 20%, rezultând o adâncime practică de minimum 132cm.*” Diferența nu este modelul — este calitatea contextului injectat.

3.6 Streaming SSE

3.6.1 Server-Sent Events — Streaming UI Asincron

Modelele LLM generează răspunsuri cu latență semnificativă, între 5 și 15 secunde pentru un răspuns complet. Într-o arhitectură HTTP clasică request-response, utilizatorul ar vedea un indicator de loading fix timp de 15 secunde, fără niciun feedback vizual — o experiență de utilizare inacceptabilă.

Server-Sent Events (SSE) rezolvă această problemă prin transformarea unui request HTTP într-un canal de comunicare unidirecțional persistent, de la server la client. Textul este transmis progresiv, în fragmente de câteva cuvinte, imediat ce modelul le generează. Experiența vizuală este identică cu cea a platformelor ChatGPT sau Claude. SSE a fost ales față de WebSocket deoarece comunicarea necesară este strict unidirecțională (server → client), iar SSE funcționează peste HTTP/1.1 standard fără upgrade de protocol, reducând complexitatea de implementare.

```
1 // aiController.ts - implementarea streaming SSE
2 export const chatStream = async (req: Request, res: Response) =>
3   ↪{
4     res.setHeader('Content-Type', 'text/event-stream');
5     res.setHeader('Cache-Control', 'no-cache');
6     res.setHeader('Connection', 'keep-alive');
7
8     const { projectId, message, history } = req.body;
9
10    // Verificare ownership - previne vulnerabilitatea IDOR
11    const project = await projectRepository.findById(projectId, req
12      ↪.user.id);
13    if (!project) return res.status(403).end();
14
15    const prompt = await buildSystemPrompt('chat', project, message
16      ↪);
17    const stream = await geminiClient.generateContentStream([
18      ...history,
19      { role: 'user', parts: [{ text: message }] }
20    ]);
21
22    for await (const chunk of stream.stream) {
23      res.write(`data: ${JSON.stringify({ text: chunk.text() })}\n\n
24        ↪`);
25    }
26
27    res.write(`data: [DONE]\n\n`);
28    res.end();
29  }
```

Verificarea ownership-ului înainte de deschiderea stream-ului este o măsură de securitate critică. Fără aceasta, orice utilizator autentificat ar putea accesa conversațiile AI ale oricărui alt utilizator modificând `projectId` în request.

3.6.2 Managementul Memoriei Conversaționale

Modelele AI nu au memorie intrinsecă între request-uri — fiecare apel API este independent. Pentru a simula o conversație continuă, istoricul mesajelor trebuie trimis la fiecare request nou. Aceasta generează două probleme pe termen lung: costul crește liniar cu numărul de mesaje (fiecare token trimis are cost), iar la un moment dat conversația depășește fereastra de context a modelului.

Soluția implementată este rezumarea periodică a conversației. Modulul `summarizeConversation` trimite ultimele `N` mesaje modelului `gemini-flash` — un model mai rapid și mai ieftin,

suficient pentru această sarcină non-creativă — cu instrucțiunea de a genera un rezumat concis în 150-200 de cuvinte. Valoarea `temperature: 0.1` asigură determinism maxim: un rezumat tehnic trebuie să fie fidel, nu creativ.

Rezumatul este persistat în baza de date prin operația `upsert` (`update or insert`), care garantează un singur rezumat activ per combinație `projectId + phase + screen`. La revenirea utilizatorului pe un ecran vizitat anterior, rezumatul este încărcat din baza de date și injectat în prompt-ul de sistem, restaurând contextul conversației fără a retrimite toate mesajele originale. Această tehnică se numește *Context Rehydration* și este esențială pentru menținerea consistenței stării în aplicații cu conversații persistente.

Legarea directă a sesiunii de `projectId` permite și o optimizare importantă: injectarea automată a contextului proiectului în fiecare request AI, fără ca utilizatorul să re-descrie terenul sau configurația casei la fiecare sesiune nouă.

3.7 Wizard-ul de Configurare — Salvarea Progresivă

3.7.1 Principiul DB ca Sursă de Adevăr

Wizard-ul de configurare în 4 pași implementează un principiu arhitectural fundamental: baza de date este sursa de adevăr pentru starea wizard-ului, nu state-ul local React. La fiecare completare a unui pas, frontend-ul trimite un `PATCH /api/projects/:id` cu datele colectate și `wizardStep` incrementat. La refresh de pagină sau la revenirea pe o sesiune nouă, aplicația citește `wizardStep` din baza de date și restaurează utilizatorul exact la pasul unde s-a oprit — datele introduse nu se pierd niciodată.

Toate actualizările trec printr-un controller care aplică un whitelist de câmpuri permise la scriere. Fără această filtrare, un utilizator malițios ar putea trimite `{ "userId": 1 }` în body și suprascrie câmpuri sensibile neexpuse în interfață — o vulnerabilitate de tip mass assignment. Doar câmpurile explicit declarate în whitelist ajung la repository.

3.7.2 Fluxul Complet al Celor 4 Pași

Pasul 1 — Identificarea Terenului: utilizatorul dă click pe hartă, coordonatele GPS sunt capturate și trimise la `POST /api/terrain/analyze-location`. Serviciul geospațial apelează Nominatim, extrage județul, caută valorile în JSON-urile normative și returnează zona seismică și adâncimea de îngheț. Datele sunt afișate imediat în UI și salvate în baza de date.

Pasul 2 — Caracteristicile Terenului: utilizatorul introduce tipul de sol, panta terenului și orientarea față de stradă. Dacă nu cunoaște tipul de sol, un widget AI — conectat la RAG din NP112-2014, Capitolul 3 — îi explică cum să identifice vizual tipul de sol și sugerează automat completarea câmpului.

Pasul 3 — Reglementările Aplicabile: numărul maxim de etaje și adâncimea minimă a fundației sunt calculate determinist din `floor-rules.json` și `frost-depth.json`, pe baza datelor din pași anteriori. Asistentul AI explică în streaming SSE de ce există aceste limite, citând articolele relevante din P100-1/2013 și NP112-2014.

Pasul 4 — Tipul Casei: utilizatorul alege stilul arhitectural și configurația etajelor. Opțiunile care depășesc numărul maxim de etaje calculat la pasul anterior sunt dezactivate vizual, cu tooltip explicativ. La finalizare, proiectul este marcat `isCompleted: true` și utilizatorul este redirecționat la pagina de detalii a proiectului.

3.8 Izolarea Datelor între Utilizatori — Tenant Security

Toate operațiunile asupra proiectelor verifică `userId` în condiția `WHERE` a fiecărui query, garantând că un utilizator nu poate accesa, modifica sau șterge proiectele altui utilizator, chiar dacă ghicește ID-ul numeric al acestora.

Această abordare implementează Row-Level Security la nivel de aplicație — echivalentul funcțional al RLS nativ PostgreSQL, fără overhead-ul de configurare al politicilor de bază de date. Middleware-ul `tenantGuard` verifică existența `req.user.id` pe toate rutele protejate înainte ca request-ul să ajungă la controller, asigurând că nicio operație de business nu se execută fără un context de identitate valid.

Bibliografie

Bibliografie

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", NeurIPS 2020. Available: <https://arxiv.org/abs/2005.11401>.
- [2] Zhouxing Gao, et al., "Retrieval-Augmented Generation for Large Language Models: A Survey", 2023. Available: <https://arxiv.org/abs/2312.10997>.
- [3] Daniel Sevilla, et al., "Compute Trends Across Three Eras of Machine Learning", 2022. Available: <https://arxiv.org/abs/2202.05924>.
- [4] Nominatim (OpenStreetMap). Available: <https://nominatim.openstreetmap.org>.
- [5] Institutul Național de Statistică (INS). Autorizări de construire. Available: https://insse.ro/cms/sites/default/files/com_presa/com_pdf/autoriz12r25.pdf.
- [6] McKinsey & Company, "The construction productivity imperative". Available: <https://www.mckinsey.com/capabilities/operations/our-insights/the-construction-productivity-imperative>.
- [7] Eurostat, "Housing in Europe 2024 Interactive Publication". Available: <https://ec.europa.eu/eurostat/web/interactive-publications/housing>.